1

ALSU_1	When the reset is asserted, the outputs should be low	Directed at the start of the simulation	bins: out_low	A checker in the testbench to make sure the output is correct
ALSU_2	Incase of invalid cases occur, Leds should blink and out bits are set to low,	Randomization under constraint where invalid cases shouldn't occur as frequent as valid cases	bins: Leds_blink, out_low, bins_invalid	A checker in the testbench to make sure the output is correct
ALSU_3	Incase of invalid cases do not occur,a and the bypass_A or bypass_B are high then the output will take the value of A or B depending on the parameter INPUT_PRIORITY	Randomization under constraint on bypass ports are disabled most of the time	bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default	A checker in the testbench to make sure the output is correct
ALSU_4	Incase of invalid cases do not occur, when opcode is ADD, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high	Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	cover the maximum, minimum and zero as It is the default value of A and B and cross coverage between these value and opcode ADD	A checker in the testbench to make sure the output is correct
ALSU_5	Incase of invalid cases do not occur, when opcode is MULT, then out should perform the multiplication on ports A and B	Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values	cover the maximum, minimum and zero as It is the default value of A and B and cross coverage between these value and opcode MULT	A checker in the testbench to make sure the output is correct
ALSU_6	Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B are low	Randomization without constraint on A or B.	bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, bins_bitwise	A checker in the testbench to make sure the output is correct
ALSU_7	Incase of invalid cases do not occur, when opcode is OR and if reduction_A or reduction_B is high, then output should perform the reduction OR operation on ports A or ports B depends on the priority	Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	cover the walking ones as It is the default value of A and B and cross coverage between these value, red_op and opcode OR	A checker in the testbench to make sure the output is correct
ALSU_8	Incase of invalid cases do not occur, when opcode is XOR, then out should perform the XOR operation on ports A and B if reduction_A and reduction_B are low	Randomization without constraint on A or B.	bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, , bins_bitwise	A checker in the testbench to make sure the output is correct
ALSU_9	Incase of invalid cases do not occur, when opcode is XOR and if reduction_A or reduction_B is high, then output should perform the reduction XOR operation on ports A or ports B depends on the priority	Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high	cover the walking ones as It is the default value of A and B and cross coverage between these value, red_op and opcode XOR	A checker in the testbench to make sure the output is correct
ALSU_10	Incase of invalid cases do not occur, when opcode is SHIFT, then out should be shifted by 1 bit depending on direction bit	Randomization without constraint	cover the SHIFT and cross coverage SHIFT, shift_in and direction	A checker in the testbench to make sure the output is correct
ALSU_11	Incase of invalid cases do not occur, when opcode is ROTATE, then out should be rotated by 1 bit depending on direction bit	Randomization without constraint	cover the ROTATE and cross coverage ROTATE and direction	A checker in the testbench to make sure the output is correct
ALSU_12	Incase of invalid cases do not occur, the output should be evaluated based on the operation	Randomization under valid input stimulus while looping on different opcode every clock cycle	bins: bins_trans	A checker in the testbench to make sure the output is correct

```
package ALSU_pkg;

    typedef enum bit [2:0] { OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID6, INVALID7 } opcode_e;
    typedef enum bit [2:0] { MAXPOS = 3'b011, ZERO = 3'b000, MAXNEG = 3'b100 } reg_e;
    parameter VALID_OP = 6;

    class ALSU_cst;
        // Declare the random variables
        rand logic cin, rst, red_op_A, red_op_B;
        rand logic bypass_A, bypass_B, direction, serial_in;

        // Declare the random variable for opcode and an array of opcodes
        rand opcode_e opcode;
        rand opcode_e opcode_valid [VALID_OP];

        // Declare the random variables for inputs A and B, and their corner cases and walking ones patterns
        rand logic [2:0] A, B;
        rand reg_e enum_ext_A, enum_ext_B;
        rand logic [2:0] A_rem_val, B_rem_val;

        bit [2:0] walking_ones[] = '{3'b100, 3'b010, 3'b001};
        rand bit [2:0] walking_ones_t, walking_ones_f;

        //1. Reset to be asserted with a low probability that you decide.
        constraint rst_cst{
            rst dist{0:= 99, 1:= 1};
        }
```

```
    //2. The inputs A and B to be mostly zero, with some random values.
    constraint A_B_cst{
        walking_ones_t inside{walking_ones}; // Constrain the walking_ones_t to be one of the patterns in walking_ones
        !(walking_ones_f inside {walking_ones}); // Constrain the walking_ones_f to not be any of the patterns in walking_ones
        !(A_rem_val inside {enum_ext_A}); // Constrain A_rem_val to not be any of the corner cases
        !(B_rem_val inside {enum_ext_B}); // Constrain B_rem_val to not be any of the corner cases

        // If the opcode is OR or XOR
        if ((opcode == OR) || (opcode == XOR)){
            // red_op_A is high, constraint the input A most of the time
            // to have one bit high in its 3 bits while constraining the B bits to be low
            if (red_op_A == 1){
                A dist {walking_ones_t := 90, walking_ones_f := 10};
                B == 3'b000;
            }
            // red_op_B is high, constraint the input B most of the time
            // to have one bit high in its 3 bits while constraining the A bits to be low
            else if (red_op_B == 1){
                B dist {walking_ones_t := 90, walking_ones_f := 10};
                A == 3'b000;
            }
        } else {
            red_op_A dist {1:= 10, 0 := 90};
            red_op_B dist {1:= 10, 0 := 90};
            // If the opcode is ADD or MULT, constraint the inputs A and B  to take the values (MAXPOS, ZERO and MAXNEG) with higher probability than the other values
            if ((opcode == ADD) || (opcode == MULT)){
            A dist {enum_ext_A := 90, A_rem_val := 10};
            B dist {enum_ext_B := 90, B_rem_val := 10};
        }
        }
    }
```

```
    // 3. The opcode to be mostly OR, XOR, ADD, MULT, ROTATE, SHIFT, with some random values for the other operations.
    // will be used while turning off opcode_arr_cst to allow random values in the opcode_arr which will be used in the covergroup to cover the transitions between al
    constraint opcode_cst{
        opcode dist{[OR:ROTATE] := 90, [INVALID6:INVALID7] := 10};
    }

    // 4. The bypass signals to be mostly disabled, with some random values.
    constraint bypass_cst{
        bypass_A dist{0:= 95, 1:= 5};
        bypass_B dist{0:= 95, 1:= 5};
    }

    // Create a fixed array of 6 elements of type opcode_e. constraint the elements of the array using
    // foreach to have a unique valid value each time randomization occurs.
    // will be used in the covergroup to cover all the operations and the transitions between them while turning off opcode_cst to allow the array to be randomized wi
    constraint opcode_seq_con{
        foreach(opcode_valid[i])
            foreach(opcode_valid[j]){
                if(i != j){
                opcode_valid[i] != opcode_valid[j];
                opcode_valid[i] inside {[OR:ROTATE]};
                }
            }
    }

    covergroup cvr_gp;
```

```
covergroup cvr_gp;

    // Cover A values (corner cases + misc)
    A_cp1: coverpoint A {
        bins A_data_0 = {3'b000};
        bins A_data_max = {3'b011};
        bins A_data_min = {3'b100};
        bins A_data_default = default;
    }

    // Walking ones when red_op_A = 1
    A_cp2: coverpoint A iff(red_op_A){
        bins A_data_walkingones[] ={3'b001, 3'b010, 3'b100};
    }

    // Cover B values (corner cases + misc)
    B_cp1: coverpoint B {
        bins B_data_0 = {3'b000};
        bins B_data_max = {3'b011};
        bins B_data_min = {3'b100};
        bins B_data_default = default;
    }

    // Walking ones when red_op_B = 1
    B_cp2: coverpoint B iff(red_op_B  && !red_op_A ) {
        bins B_data_walkingones[] ={3'b001, 3'b010, 3'b100};
    }
```

```
    // Cover the opcode with the specified bins and transitions
    ALU_cp: coverpoint opcode{
        bins bins_shift[] = {SHIFT, ROTATE};
        bins bins_arith[] = {ADD, MULT};
        bins bins_bitwise[] = {OR, XOR};
        bins bins_invalid[] = {INVALID6, INVALID7};
        bins bins_trans = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
    }

    opcode_cp: coverpoint opcode;

    cin_cp: coverpoint cin {
        bins cin_zero = {0};
        bins cin_one = {1};
    }

    dir_cp: coverpoint direction{
        bins dir_zero = {0};
        bins cin_one = {1};
    }

    shift_cp: coverpoint serial_in{
        bins shift_zero = {0};
        bins shift_one = {1};
    }

    redA_cp: coverpoint red_op_A{
        bins redA_on = {1};
        bins redA_off = {0};
    }
```

```
        redB_cp: coverpoint red_op_B{
            bins redB_on = {1};
            bins redB_off = {0};
        }

        add_mult_AB_cs : cross opcode_cp, A_cp1, B_cp1{
            option.cross_auto_bin_max = 0;
            bins add_mult_A_B = binsof(opcode_cp) intersect{ADD,MULT} &&
                                binsof(A_cp1) intersect{MAXPOS, MAXNEG, ZERO} &&
                                binsof(B_cp1) intersect{MAXPOS, MAXNEG, ZERO};
        }

        opcode_cin_cs : cross opcode_cp, cin_cp{
            option.cross_auto_bin_max = 0;
            bins opcode_cin = binsof(opcode_cp) intersect{ADD} &&
                              binsof(cin_cp);
        }

        opcode_dir_cs : cross opcode_cp, dir_cp{
            option.cross_auto_bin_max = 0;
            bins opcode_dir = binsof(opcode_cp) intersect{SHIFT, ROTATE} &&
                              binsof(dir_cp);
        }

        opcode_shift_cs : cross opcode_cp, shift_cp{
            option.cross_auto_bin_max = 0;
            bins opcode_shift = binsof(opcode_cp) intersect{SHIFT} &&
                              binsof(shift_cp);
        }
```

```
        opcode_redA_walkA_cs : cross opcode_cp, redA_cp, A_cp2, B_cp1{
            option.cross_auto_bin_max = 0;
            bins opcode_redA_walkA = binsof(opcode_cp) intersect{OR,XOR} &&
                                     binsof(redA_cp.redA_on) &&
                                     binsof(A_cp2) &&
                                     binsof(B_cp1.B_data_0);
        }

        opcode_redB_walkB_cs : cross opcode_cp, redB_cp, B_cp2, A_cp1{
            option.cross_auto_bin_max = 0;
            bins opcode_redB_walkB = binsof(opcode_cp) intersect{OR,XOR} &&
                                     binsof(redB_cp.redB_on) &&
                                     binsof(B_cp2) &&
                                     binsof(A_cp1.A_data_0);
        }

        invalid_case_cs : cross opcode_cp, redA_cp, redB_cp{
            option.cross_auto_bin_max = 0;
            bins invalid_case = !binsof (opcode_cp) intersect{OR,XOR} &&
                                (binsof(redB_cp.redB_on) || binsof(redA_cp.redA_on));
        }

    endgroup

    function new();
        cvr_gp = new();
    endfunction

endclass
```

```
import ALSU_pkg::*;

module ALSU_tb;

    // ─── 1. Parameters, classes, wires and regs ──────────────────────────

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    bit clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    bit signed [2:0] A, B;
    bit [2:0] opcode;
    logic [15:0] leds;
    bit [15:0] leds_exp;
    logic signed [5:0] out;
    bit signed [5:0] out_exp;

    // Registered stimulus used by the golden model
    bit red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
    bit signed [1:0] cin_reg;
    bit [2:0] opcode_reg;
    bit signed [2:0] A_reg, B_reg;


    int error_count = 0;
    int correct_count = 0;


    ALSU_cst ALSU_obj = new();
```

```
    // ─── 2. DUT instantiation ────────────────────────────────────────────
    ALSU #(.INPUT_PRIORITY(INPUT_PRIORITY), .FULL_ADDER(FULL_ADDER)) uut(
        .clk(clk),
        .cin(cin),
        .rst(rst),
        .red_op_A(red_op_A),
        .red_op_B(red_op_B),
        .bypass_A(bypass_A),
        .bypass_B(bypass_B),
        .direction(direction),
        .serial_in(serial_in),
        .opcode(opcode),
        .A(A),
        .B(B),
        .leds(leds),
        .out(out)
    );

    // ─── 3. Clock generation ─────────────────────────────────────────────
    initial begin
        clk = 1'b0;
        forever begin
            #5 clk = ~clk;
            ALSU_obj.clk = clk;
        end
    end
```

```
// ─── 4. Stimulus generation ──────────────────────────────────────────
initial begin
    #5;
    // assert reset
    assert_reset;

    // ── First Loop: constraints 1-7 active, constraint 8 disabled ──────
    $display("___________________First Loop_______________________");
    ALSU_obj.opcode_arr_cst.constraint_mode(0);

    repeat(1000) begin
        assert(ALSU_obj.randomize());
        cin = ALSU_obj.cin;
        rst = ALSU_obj.rst;
        red_op_A = ALSU_obj.red_op_A;
        red_op_B = ALSU_obj.red_op_B;
        bypass_A = ALSU_obj.bypass_A;
        bypass_B = ALSU_obj.bypass_B;
        direction = ALSU_obj.direction;
        serial_in = ALSU_obj.serial_in;
        opcode = ALSU_obj.opcode;
        A = ALSU_obj.A;
        B = ALSU_obj.B;
        if(ALSU_obj.rst) begin
            check_reset();
        end else begin
            ALSU_obj.sample_coverage();
            golden_model();
            check_result();
        end
    end

    // ── Between loops: disable all constraints, force signals to 0 ──────
    ALSU_obj.constraint_mode(0); // disable constraints 1-7
    ALSU_obj.rst.rand_mode(0);
    ALSU_obj.red_op_A.rand_mode(0);
    ALSU_obj.red_op_B.rand_mode(0);
    ALSU_obj.bypass_A.rand_mode(0);
    ALSU_obj.bypass_B.rand_mode(0);
    ALSU_obj.opcode_arr_cst.constraint_mode(1); // enable constraint 8
```

```
    // ── Second Loop: unique opcode sequence, randomise other inputs ────────
    $display("_________________Second Loop_______________________");
    repeat(10) begin
        assert(ALSU_obj.randomize());

        // Apply inputs that stay constant across the 6 inner iterations
        cin = ALSU_obj.cin;
        direction = ALSU_obj.direction;
        serial_in = ALSU_obj.serial_in;
        A = ALSU_obj.A;
        B = ALSU_obj.B;

        // Iterate over the 6 unique valid opcodes
        for (int i = 0; i < 6; i++) begin
            opcode = ALSU_obj.opcode;
            ALSU_obj.sample_coverage();
            golden_model();
            check_result();
        end
    end


    $display("Total Correct: %0d, Total Errors: %0d", correct_count, error_count);
    $stop;
end

 // ─── Assert and check reset ──────────────────────────────────────────
task assert_reset;
    rst = 1'b1;
    @(negedge clk);
    check_reset;
    rst = 1'b0;
endtask
```

```
// Task to check reset behavior
task check_reset;
    @(negedge clk);
    if ((leds !== 0) || (out !== 0)) begin
        error_count += 1;
        $display("%t: ERROR for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
    end else begin
        correct_count += 1;
        $display("%t: SUCCESS for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
        reset_internals();
    end
endtask

task reset_internals();
    {red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg} = 6'b00_0000;
    cin_reg = 2'b00;
    opcode_reg = 3'b000;
    {A_reg,B_reg} = 6'b000000;
    leds_exp   = 16'd0;
    out_exp    = 6'd0;
endtask

// ─── Validity check (mirrors DUT logic) ──────────────────────────────
function bit is_invalid();
    bit invalid;
    if((red_op_A_reg || red_op_B_reg) && (opcode_reg > 3'b001))
        invalid = 1'b1;
    else if (opcode_reg == INVALID6 || opcode_reg == INVALID7)
        invalid = 1'b1;
    else
        invalid = 1'b0;
    return invalid;
endfunction
```

```
// Task to implement the golden model and check results
task golden_model ();
    if (is_invalid()) begin
        leds_exp = ~leds_exp;
    end
    else begin
        leds_exp = 0;
    end

    if(bypass_A_reg)
        out_exp = A_reg;
    else if(bypass_B_reg)
        out_exp = B_reg;

    else if(is_invalid())
        out_exp = 0;

    else begin
        if(opcode_reg == OR) begin
            if(red_op_A_reg)
                out_exp = |A_reg;
            else if (red_op_B_reg)
                out_exp = |B_reg;
            else
                out_exp = A_reg | B_reg;
        end

        else if(opcode_reg == XOR) begin
            if(red_op_A_reg)
                out_exp = ^A_reg;
            else if (red_op_B_reg)
                out_exp = ^B_reg;
            else
                out_exp = A_reg ^ B_reg;
        end
```

```
            else if(opcode_reg == ADD)
                out_exp = A_reg + B_reg + cin_reg;

            else if(opcode_reg == MULT)
                out_exp = A_reg * B_reg;

            else if (opcode_reg == SHIFT) begin
                if(direction_reg)
                    out_exp = {out_exp[4:0], serial_in_reg};
                else
                    out_exp = {serial_in_reg, out_exp[5:1]};
            end

            else if (opcode_reg == ROTATE) begin
                if(direction_reg)
                    out_exp = {out_exp[4:0], out_exp[5]};
                else
                    out_exp = {out_exp[0], out_exp[5:1]};
            end
        end
        update_internals();
    endtask

    task update_internals();
        cin_reg = cin;
        red_op_A_reg = red_op_A;
        red_op_B_reg = red_op_B;
        bypass_A_reg = bypass_A;
        bypass_B_reg = bypass_B;
        direction_reg = direction;
        serial_in_reg = serial_in;
        opcode_reg = opcode;
        A_reg = A;
        B_reg = B;
    endtask
```

```
    // Task to check results against the golden model
    task check_result();
        @(negedge clk);
        if ((out_exp !== out) || (leds_exp !== leds)) begin
            error_count += 1;
            $display("%t: ERROR--> A = %0d B = %0d opcode = %0d red_op_A = %0d, red_op_B = %0d bypass_A_reg = %0d, bypass_B_reg = %0d direction_reg = %0d serial_in_reg = %0d",
                     $time, A, B, opcode, red_op_A, red_op_B, bypass_A, bypass_B,
                     direction, serial_in);
            $display("out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                     out, out_exp, leds, leds_exp);

        end
        else begin
            correct_count += 1;
             $display("%t: SUCCESS--> A = %0d B = %0d opcode = %0d opcode_arr = %0p red_op_A = %0d, red_op_B = %0d bypass_A_reg = %0d, bypass_B_reg = %0d direction_reg = %0d serial_in_reg = %0d",
                     $time, A, B, opcode,ALSU_obj.opcode_arr, red_op_A, red_op_B, bypass_A, bypass_B,
                     direction, serial_in);
            $display("out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                     out, out_exp, leds, leds_exp);
        end
    endtask

endmodule
```

```
================================================================================
=== Instance: /ALSU_tb/uut
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        31        31         0   100.00%

================================Branch Details================================
```

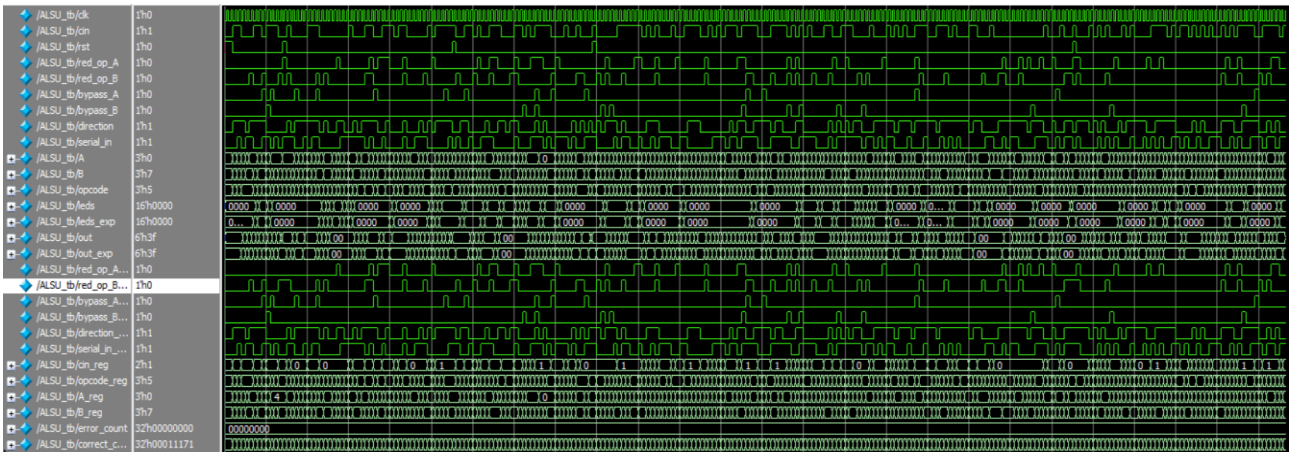
```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       6         6         0   100.00%

================================Condition Details================================
```

```
Expression Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Expressions                      8         8         0   100.00%

================================Expression Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        116       116         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /ALSU_tb/uut --
```

INST \/ALSU_pkg::ALSU_cst::cvr_gp	100.00%	100	100.00%
CVP A_cp1	100.00%	100	100.00%
CVP A_cp2	100.00%	100	100.00%
CVP B_cp1	100.00%	100	100.00%
CVP B_cp2	100.00%	100	100.00%
CVP ALU_cp	100.00%	100	100.00%
CVP opcode_cp	100.00%	100	100.00%
CVP cin_cp	100.00%	100	100.00%
CVP dir_cp	100.00%	100	100.00%
CVP shift_cp	100.00%	100	100.00%
CVP redA_cp	100.00%	100	100.00%
CVP redB_cp	100.00%	100	100.00%
CROSS add_mult_AB_cs	100.00%	100	100.00%
CROSS opcode_cin_cs	100.00%	100	100.00%
CROSS opcode_dir_cs	100.00%	100	100.00%
CROSS opcode_shift_cs	100.00%	100	100.00%
CROSS opcode_redA_walkA_cs	100.00%	100	100.00%
CROSS opcode_redB_walkB_cs	100.00%	100	100.00%
CROSS invalid_case_cs	100.00%	100	100.00%



2

```
module test();

    logic a, b, c;
    logic clk;
    logic [7:0] Y;
    logic [3:0] D;
    logic valid;

    // Write assert property statement, if signal a is high in a positive edge of a clock then signal b
    // should be high after 2 clock cycles
    property ab_high;
        @(posedge clk) $rose(a) |-> ##2 b;
    endproperty
    // Write assert property statement, If signal a is high and signal b is high then signal c should be
    // high 1 to 3 clock cycle later
    property ab_c;
        @(posedge clk) (a && b) |-> ##[1:3] c;
    endproperty

    // Write a sequence s11b, after 2 positive clock edges, signal b should be low
    sequence s11b;
        ##2 !b;
    endsequence

    // Write a property for the following specs:
    // - 3-to-8 decoder output Y
    // i. At each positive edge of clock, Y output must be only one bit high
    property Y_onebit;
        @(posedge clk) $onehot(Y);
    endproperty
```

```
    // 4-to-2 priority encoder output valid (refer to assignment 1)
    // i. At each positive edge of clock, if the input D bits are low then after one clock
    // cycle, output valid must be low.
    property pri_encoder;
        @(posedge clk) (|D == 0) |-> ##1 !valid;
    endproperty
    always @(posedge clk) begin

        assert property (ab_high);
        assert property (ab_c);
        assert property (Y_onebit);
        assert property (pri_encoder);

    end

endmodule
```

3-

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: Counter Design
//
////////////////////////////////////////////////////////////////////////////////
module counter_top();

    // 1. Generate the clock
    bit clk;
    initial begin
    clk = 0;
    forever #5 clk = ~clk;
    end

    // 2. instantiate the interface, and pass the clock
    counter_if c_if(clk);

    // 3. instantiate the tb, DUT, monitor, and pass the interface
    counter_tb tb(c_if);

    counter dut(c_if);

    counter_monitor monitor(c_if);

    // 4. bind the SVA module to the design, and pass the interface
    bind counter counter_sva counter_sva_inst(c_if);

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: Counter Design
//
////////////////////////////////////////////////////////////////////////////////
module counter (counter_if.DUT c_if);

always @(posedge c_if.clk or negedge c_if.rst_n) begin
    if (!c_if.rst_n)
        c_if.count_out <= 0;
    else if (!c_if.load_n)
        c_if.count_out <= c_if.data_load;
    else if (c_if.ce)
        if (c_if.up_down)
            c_if.count_out <= c_if.count_out + 1;
        else
            c_if.count_out <= c_if.count_out - 1;
end

assign c_if.max_count = (c_if.count_out == {c_if.WIDTH{1'b1}})? 1:0;
assign c_if.zero = (c_if.count_out == 0)? 1:0;

endmodule
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: Counter Design
//
////////////////////////////////////////////////////////////////////////////////
interface counter_if(clk);

    // 1. Add the parameters
    parameter WIDTH = 4;

    // 2. Add the clock as an input port
    input clk;

    // 3. Add the internal signals of the interface
    logic rst_n;
    logic load_n;
    logic up_down;
    logic ce;
    logic [WIDTH-1:0] data_load;
    logic [WIDTH-1:0] count_out;
    logic  max_count;
    logic  zero;

    // 4. Add the modports
    modport DUT(input clk, rst_n, load_n, up_down, ce, data_load, output count_out, max_count, zero);

    modport TEST (output rst_n, load_n, up_down, ce, data_load, input clk, count_out, max_count, zero);

    modport MONITOR (input clk, rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);


endinterface
```

```
////////////////////////////////////////////////////////////////////////////////
// Author: Mohamed Ayman
// Course: Digital Verification using SV & UVM
//
// Description: Counter Design
//
////////////////////////////////////////////////////////////////////////////////
module counter_monitor(counter_if.MONITOR c_if);

    initial begin
        $monitor("rst_n = %b, load_n = %b, up_down = %b, ce = %b, data_load = %b, count_out = %b, max_count = %b, zero = %b",
                 c_if.rst_n, c_if.load_n, c_if.up_down, c_if.ce, c_if.data_load, c_if.count_out, c_if.max_count, c_if.zero);
    end

endmodule
```

```
module counter_sva(counter_if.DUT c_if);

    // When the load control signal is active, then the dout has the value of the din
    property  load_active;
        @(posedge c_if.clk) disable iff(!c_if.rst_n)  !c_if.load_n |=> (c_if.count_out == $past(c_if.data_load));
    endproperty

    a_load_active_assertion: assert property(load_active);

    a_load_active_coverage: cover property(load_active);

    // When the load control signal is not active, and the enable is off then the dout does not change
    property  load_inactive_en_off;
        @(posedge c_if.clk) disable iff(!c_if.rst_n) (c_if.load_n && !c_if.ce) |=> $stable(c_if.count_out);
    endproperty

    a_hold_assertion: assert property(load_inactive_en_off);

    a_hold_coverage: cover property(load_inactive_en_off);

    // When the load control signal is not active and the enable is active, and the up_down is high
    // then the dout is incremented.
    property  load_inactive_en_on_up;
        @(posedge c_if.clk) disable iff(!c_if.rst_n) (c_if.load_n && c_if.ce && c_if.up_down) |=>  c_if.count_out == $past(c_if.count_out) + 1'b1;
    endproperty

    a_increment_assertion: assert property(load_inactive_en_on_up);

     a_increment_coverage: cover property(load_inactive_en_on_up);
```

```
    property  load_inactive_en_on_down;
        @(posedge c_if.clk) disable iff(!c_if.rst_n) (c_if.load_n && c_if.ce && !c_if.up_down)|=>  c_if.count_out == $past(c_if.count_out) - 1'b1;
    endproperty

    a_decrement_assertion: assert property(load_inactive_en_on_down);

    a_decrement_coverage: cover property(load_inactive_en_on_down);

    property async_reset;
        @(posedge c_if.clk) !c_if.rst_n |=>  (c_if.count_out == 0);
    endproperty

    a_reset_assertion: assert property(async_reset);

    a_reset_coverage: cover property(async_reset);

    property max_count_check;
        @(posedge c_if.clk) disable iff(!c_if.rst_n) (c_if.count_out== {c_if.WIDTH{1'b1}}) |=>  $past(c_if.max_count);
    endproperty

    a_max_count_assertion: assert property(max_count_check);

    a_max_count_coverage: cover property(max_count_check);

    property zero_check;
    @(posedge c_if.clk) (c_if.count_out == 0) |=> $past(c_if.zero);
    endproperty

    a_zero_assertion: assert property(zero_check);

    a_zero_coverage: cover property(zero_check);

endmodule
```

```
module counter_tb (counter_if.TEST c_if);

    //counter_class counter_transaction;

    // reset initial values for inputs
    initial begin
        //counter_transaction = new();
        c_if.rst_n = 1;
        c_if.load_n = 1;
        c_if.up_down = 0;
        c_if.ce = 0;
        c_if.data_load = 0;

        // Reset
        repeat(2) @(posedge c_if.clk);
        c_if.rst_n = 0;

        repeat(2) @(posedge c_if.clk);
        c_if.rst_n = 1;

        // load test
        @(posedge c_if.clk);
        c_if.load_n = 0; c_if.data_load = 4'b0001;

        @(posedge c_if.clk);
        c_if.data_load = 4'b0010;

        @(posedge c_if.clk);
        c_if.data_load = 4'b0100;

        @(posedge c_if.clk);
        c_if.data_load = 4'b1111;
```

```
        // Hold test
        @(posedge c_if.clk);
        c_if.load_n = 1;
        repeat(3) @(posedge c_if.clk);

        //Up count
        @(posedge c_if.clk);
        c_if.load_n = 1; c_if.ce = 1; c_if.up_down = 1;

        repeat(15) @(posedge c_if.clk);

        @(posedge c_if.clk);
        c_if.ce = 0;

        // load test
        @(posedge c_if.clk);
        c_if.load_n = 1; c_if.data_load = 4'b0000;
        repeat(2) @(posedge c_if.clk);

        // // Down count
        @(posedge c_if.clk);
        c_if.ce = 1; c_if.up_down = 0;

        repeat(15) @(posedge c_if.clk);

        @(posedge c_if.clk);
        c_if.ce = 0;

        $stop;
    end
endmodule
```

```
DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                        Design Design   Lang File(Line)      Hits Status
                                            Unit   UnitType
--------------------------------------------------------------------------------------------
/counter_top/dut/counter_sva_inst/a_load_active_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(17)
                                                                                     4 Covered
/counter_top/dut/counter_sva_inst/a_hold_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(26)
                                                                                    10 Covered
/counter_top/dut/counter_sva_inst/a_increment_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(36)
                                                                                    16 Covered
/counter_top/dut/counter_sva_inst/a_decrement_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(46)
                                                                                    14 Covered
/counter_top/dut/counter_sva_inst/a_reset_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(54)
                                                                                     2 Covered
/counter_top/dut/counter_sva_inst/a_max_count_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(62)
                                                                                    10 Covered
/counter_top/dut/counter_sva_inst/a_zero_coverage
                                            counter_sva Verilog  SVA  counter_sva.sv(70)
                                                                                     5 Covered

TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 7
```

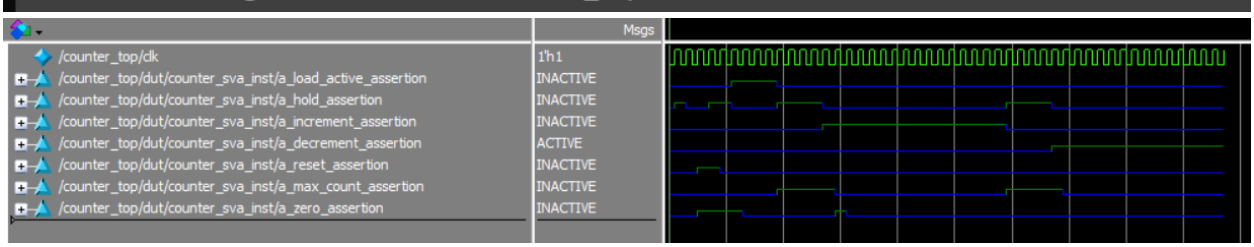
```
ASSERTION RESULTS:
--------------------------------------------------------------------
Name                File(Line)                    Failure      Pass
                                                  Count        Count
--------------------------------------------------------------------
/counter_top/dut/counter_sva_inst/a_load_active_assertion
                    counter_sva.sv(15)                 0          1
/counter_top/dut/counter_sva_inst/a_hold_assertion
                    counter_sva.sv(24)                 0          1
/counter_top/dut/counter_sva_inst/a_increment_assertion
                    counter_sva.sv(34)                 0          1
/counter_top/dut/counter_sva_inst/a_decrement_assertion
                    counter_sva.sv(44)                 0          1
/counter_top/dut/counter_sva_inst/a_reset_assertion
                    counter_sva.sv(52)                 0          1
/counter_top/dut/counter_sva_inst/a_max_count_assertion
                    counter_sva.sv(60)                 0          1
/counter_top/dut/counter_sva_inst/a_zero_assertion
                    counter_sva.sv(68)                 0          1

Total Coverage By Instance (filtered view): 99.70%
```

```
================================================================================
=== Instance: /counter_top/c_if
=== Design Unit: work.counter_if
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================
```

```
================================================================================
=== Instance: /counter_top/dut
=== Design Unit: work.counter
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        10        10         0   100.00%

================================Branch Details================================
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       2         2         0   100.00%

================================Condition Details=============================

Condition Coverage for instance /counter_top/dut --
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details=============================

Statement Coverage for instance /counter_top/dut --
```



4

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reg_file_1	When the reset is asserted, The output should equal to the the reset values according to the provided table	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
Reg_file_2	When the write is disabled,the register value must not change	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
Reg_file_3	When the write is asserted, the data_in wil be written in the appropriate register based on the given address	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
Reg_file_4	Address Decoding check by writing to one register and read from another to ensure no unintended modification	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
Reg_file_5	write to registers and Read back using data out and compare expected value	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct

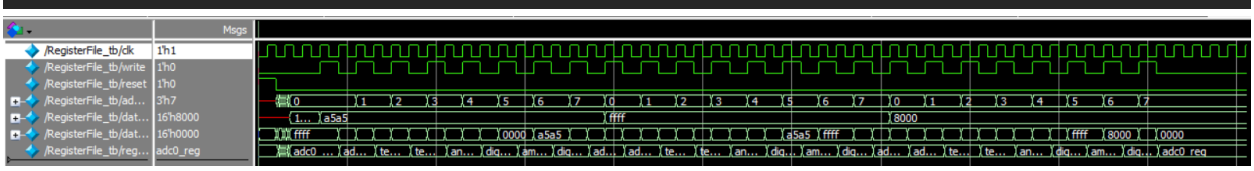
```
typedef enum {  adc0_reg   = 0    ,
                adc1_reg          ,
                temp_sensor0_reg ,
                temp_sensor1_reg ,
                analog_test       ,
                digital_test      ,
                amp_gain          ,
                digital_config} reg_file;


module RegisterFile_tb();

    // inputs and outputs
    logic clk, write ,reset;
    logic [2:0] address;
    logic [15:0] data_in;
    logic [15:0] data_out;

    // DUT
    config_reg DUT(
        clk,
        reset,
        write,
        data_in,
        address,
        data_out
    );

    reg_file reg_selection;
```

```
always #5 clk = ~clk;

// task for reset
task reset_assert;
    reset = 1'b1;
    write = 1'b0;
    #10;
    reset = 1'b0;
endtask

// task for check
task check_result(string name, input [15:0] expected_res);
    if(data_out !== expected_res) begin
        $display("ERROR in %s: Expected = %h, Got = %h", name, expected_res, data_out);
    end else begin
        $display("PASS in %s: Value = %h", name, data_out);
    end
endtask

// task for write
task write_reg(reg_file reg_r, logic [15:0] val);
    @(posedge clk);
    address = reg_r;
    data_in = val;
    write = 1'b1;
    @(posedge clk);
    write = 1'b0;
endtask
```

```
// general stimuli
initial begin
    clk = 0;

    reset_assoc["adc0_reg"] = 16'hFFFF;
    reset_assoc["adc1_reg"] = 16'h0;
    reset_assoc["temp_sensor0_reg"] = 16'h0;
    reset_assoc["temp_sensor1_reg"] = 16'h0;
    reset_assoc["analog_test"] = 16'hABCD;
    reset_assoc["digital_test"] = 16'h0;
    reset_assoc["amp_gain"] = 16'h0;
    reset_assoc["digital_config"] = 16'h1;

    // assert reset
    reset_assert();

    $display("----------------------Check Reset assertion----------------------");
    // check reset
    reg_selection = reg_selection.first();
    repeat(reg_selection.num()) begin
        address = reg_selection;
        #1;
        check_result(reg_selection.name(), reset_assoc[reg_selection.name()]);
        reg_selection = reg_selection.next();
    end

    $display("----------------------Check write disable----------------------");
    // write disable Test
    reg_selection = reg_selection.first();
        address = adc0_reg;
        data_in = 16'h1234;
        write = 0;
        #10;
        check_result("adc0_reg", 16'hFFFF);
```

```
    $display("----------------------Check write 1----------------------");
    // write and read
    reg_selection = reg_selection.first();
    repeat(reg_selection.num()) begin
        write_reg(reg_selection, 16'hA5A5);

        address = reg_selection;
        #1;

        check_result(reg_selection.name(), 16'hA5A5);
        reg_selection = reg_selection.next();
    end

    $display("----------------------Check write 2----------------------");
    // write and read
    reg_selection = reg_selection.first();
    repeat(reg_selection.num()) begin
        write_reg(reg_selection, 16'hFFFF);

        address = reg_selection;
        #1;

        check_result(reg_selection.name(), 16'hFFFF);
        reg_selection = reg_selection.next();
    end
```

```
    $display("----------------------Check write 3----------------------");
    // write and read
    reg_selection = reg_selection.first();
    repeat(reg_selection.num()) begin
        write_reg(reg_selection, 16'h8000);

        address = reg_selection;
        #1;

        check_result(reg_selection.name(), 16'h8000);
        reg_selection = reg_selection.next();
    end

    #50; $stop;

end
```

```
# ---------------Bug1---------------
a) Design Input for Bug to Appear:
        1- reset register "analog_test"
b) Expected Behavior:
         expected value = 16'habcd
c) Observed Behavior
         Bit 0 of register analog_test cannot be written to a 1

# ---------------Bug2---------------
a) Design Input for Bug to Appear:
        1- reset register "temp_sensor0_reg"
b) Expected Behavior:
         Write 16'hA5A5
         Write 16'hFFFF
         Write 16'h8000
c) Observed Behavior
         A5A5 → 4B4A
         FFFF → FFFE
         8000 → 0000
# ---------------Bug3---------------
a) Design Input for Bug to Appear:
        1- reset register "digital_config"
b) Expected Behavior:
         expected value = 16'ha5a5
c) Observed Behavior
         Bit 15 of register digital_config cannot be written to a 1
```

```
# ---------------Bug4---------------
a) Design Input for Bug to Appear:
        1- reset register "digital_test"
b) Expected Behavior:
         expected value = 16'hffff
c) Observed Behavior
         output is registered for one more clock cycle

# ---------------Bug5---------------
a) Design Input for Bug to Appear:
        1- reset register "adc1_reg"
b) Expected Behavior:
         expected value = 16'h8000
c) Observed Behavior
         s are shifted / wrong bit mapping (output -> 16'h0080)
```